

INFORMÁTICA

[Área personal](#) / [Mis cursos](#) / [\(29185\) INFORMÁTICA](#) / [Tema 7. Recursión y backtracking](#) / [Lab 7. Recursión y backtracking](#)

Lab 7. Recursión y backtracking

Resuelve todos los ejercicios en la misma entrega. Esta página realizará unas pruebas básicas y no permitirá entregas con errores sintácticos o con demasiados fallos de funcionamiento.

1. Slice de máximo valor

Considera la aplicación de la técnica divide y vencerás sobre el siguiente problema. Dado un array de N números enteros, buscamos la cadena de n celdas consecutivas en este array cuya suma sea máxima (obviamente $n < N$). ¿Es conveniente aplicar divide y vencerás en este caso? ¿Cómo sería una resolución directa del problema? Implementa la función `max_slice(L)` que devuelve una tupla (a, b) de manera que `L[a:b]` sea la cadena a la que se refiere el enunciado.

Nota: Ejercicio extraído de [esta página](#).

2. Salir del laberinto

Un laberinto puede modelarse como un grafo donde cada intersección es un nodo del grafo y cada pasillo es un arco que conecta dos intersecciones. La salida del laberinto se considera el nodo $\emptyset$. El grafo se modela como una tupla de arcos. Cada arco se representa con una tupla (a, b) donde a y b son los nodos conectados. El grafo es *no dirigido*, ambos sentidos de recorrido de los arcos están permitidos. Cada nodo se representa con un número entero.

Escribe una función `maze_solver(G, I)` que recibe como argumentos un grafo y un nodo inicial, y devuelve la secuencia de arcos que determina el camino de salida. Es decir, una secuencia conectada de arcos de G que tiene como nodo origen I y como nodo destino el nodo $\emptyset$.

Ejemplo

```
>>> G = ((0,1),(1,2),(2,3),(1,3))
>>> maze_solver(G, 3)
((3,1),(1,0))
```

3. Todas las sumas

Implementa la función `sumas(n)` que devuelve todas las secuencias de números naturales que suman n .

Ejemplo

```
>>> list(sumas(4))
[(1,1,1,1),(1,1,2),(1,3),(2,2),(4,)]
```

4. Planificar backups

Disponemos de una serie de ficheros mp3 (versiones no piratas) y una caja de CD vírgenes (con canon) en los que grabarlos. El problema es que no todos los ficheros caben en los discos, y los ficheros no se pueden partir en trozos. El objetivo es encontrar una forma de grabar los ficheros en los discos de manera que se maximice el número de ficheros grabados. Resolver el problema por *backtracking* implementando la función `planifica(L, m, k)` donde L es una secuencia con los tamaños de los archivos $(t_1, t_2, \dots, t_n)$, m es el número de discos, todos ellos con la misma capacidad k . Debe devolver una secuencia de con los índices de los archivos que van en cada disco. Por ejemplo:

Ejemplo

```
>>> list(planifica([10,15,20,8], 2, 25))
[(0,1),(2,)]
```

Nota: Ejercicio extraído de [esta página](#).

5. Tarros necesarios

Este problema está muy relacionado con el anterior. El problema del empaquetamiento en recipientes consiste en lo siguiente. Tenemos n objetos de pesos $w_1, w_2, \dots, w_n$ y un número ilimitado de recipientes con capacidad máxima R (siendo $w_i \leq R$, para todo i). Los objetos se deben meter en los recipientes sin partarlos, y sin superar su capacidad máxima. Se busca el mínimo número de recipientes

Implementa la función `num_tarros(L,R)` donde `L` es la lista de los pesos y `R` la capacidad máxima de cada tarro. Debe devolver el número mínimo de tarros necesario.

Nota: Ejercicio extraído de [esta página](#).

Pruebas

A continuación se incluyen las 5 pruebas que será necesario pasar para aceptar esta entrega. Consulta [esta página](#) para aprender cómo pueden ayudarte las pruebas a desarrollar tu propio programa.

```
# Descomenta la siguiente línea y la última para ejecutar las pruebas
#from unittest import TestCase, main
```

```
class Test(TestCase):
    def test_maxslice(self):
        self.assertEqual(max_slice([-2, 1, -3, 4, -1, 2, 1, -5, 4]), (3,7))
        self.assertEqual(max_slice([3,2,6,-1,4,5,-1,2]), (0,8))
        self.assertEqual(max_slice([2,-3,6]), (2,3))
        self.assertEqual(max_slice([2]), (0,1))

    def test_maze_solver(self):
        def check_maze_solver(G, n):
            S = maze_solver(G, n)
            edges_in_graph(S, G)
            first_node_is_n(S, n)
            last_node_is_zero(S, n)

        def edges_in_graph(S, G):
            for a,b in S:
                self.assertTrue((a,b) in G or (b,a) in G)

        def first_node_is_n(S, n):
            edges_with_node = [e for e in S if e[0] == n or e[1] == n]
            self.assertEqual(len(edges_with_node), 1, 'El primer nodo solo debe aparecer una vez')

        def last_node_is_zero(S, n):
            edges = list(S)
            for _ in enumerate(S):
                n = next_node_in_path(edges, n)
                if n == 0: break
            self.assertEqual(n, 0, 'El nodo final debería ser 0')

        def next_node_in_path(S, n):
            edges_with_node = [e for e in S if e[0] == n or e[1] == n]
            self.assertEqual(len(edges_with_node), 1, 'Debería pasar solo una vez por cada interseccion')
            a,b = edges_with_node[0]
            S.remove((a,b))
            return b if a == n else a

        G = ((0,1),(1,2),(2,3),(1,3))
        check_maze_solver(G, 3)

        G = ((3,0),)
        check_maze_solver(G, 3)

        G = ((0, 1), (0, 2), (1, 3), (2, 19), (3, 4), (3, 5), (4, 6), (5, 20), (6, 7), (7, 21), (8, 9), (8, 10), (9, 11),
(10, 22), (11, 12), (12, 13), (14, 15), (14, 16), (15, 17), (16, 23), (17, 18), (18, 24), (20, 25), (21, 26), (24, 32), (25,
33), (26, 22), (27, 28), (27, 29), (28, 30), (29, 34), (30, 31), (31, 23), (32, 35), (34, 44), (35, 50), (36, 37), (36, 38),
(37, 33), (38, 51), (39, 40), (39, 41), (40, 42), (41, 52), (42, 43), (43, 53), (44, 54), (45, 46), (45, 47), (46, 48), (47,
55), (48, 49), (49, 35), (50, 56), (51, 57), (52, 60), (53, 61), (55, 62), (57, 58), (58, 59), (59, 66), (60, 67), (61, 54),
(62, 68), (63, 64), (63, 65), (64, 56), (65, 69), (66, 73), (67, 74), (68, 79), (69, 80), (70, 71), (70, 72), (71, 66), (72,
83), (73, 84), (74, 75), (75, 76), (76, 85), (77, 78), (78, 86), (79, 87), (80, 81), (81, 82), (82, 88), (83, 89), (84, 90),
(85, 93), (86, 94), (87, 95), (88, 98), (89, 99), (90, 100), (91, 92), (92, 101), (93, 102), (94, 103), (95, 104), (96, 97),
(97, 105), (98, 106), (99, 107), (100, 108), (100, 109), (102, 110), (103, 111), (104, 112), (105, 113), (107, 114), (108,
101), (109, 115), (110, 116), (111, 117), (112, 118), (113, 106), (114, 119), (117, 123), (118, 124), (119, 130), (120,
121), (120, 122), (121, 116), (122, 131), (123, 132), (124, 133), (125, 126), (125, 127), (126, 128), (127, 134), (128,
129), (129, 135), (130, 136), (130, 137), (131, 140), (132, 143), (133, 144), (134, 145), (135, 146), (136, 138), (137,
147), (138, 139), (139, 148), (140, 141), (141, 142), (142, 132), (143, 149), (144, 150), (145, 151), (146, 152), (147,
153), (148, 154), (150, 159), (150, 160), (152, 161), (153, 162), (154, 155), (155, 156), (156, 157), (157, 158), (158,
149), (159, 151), (160, 163), (161, 164), (163, 174), (164, 176), (165, 166), (165, 167), (166, 168), (167, 177), (168,
169), (169, 170), (170, 171), (171, 172), (172, 173), (173, 178), (174, 175), (176, 179), (178, 186), (179, 192), (180,
181), (181, 177), (182, 183), (183, 184), (184, 185), (185, 193), (186, 194), (187, 188), (187, 189), (188, 190), (189,
195), (190, 191), (191, 196), (192, 197), (193, 205), (194, 206), (196, 207), (197, 208), (198, 199), (198, 200), (199,
201), (200, 209), (201, 202), (202, 203), (203, 204), (204, 193), (205, 210), (206, 195), (207, 197), (208, 211), (209,
212), (212, 225), (213, 214), (214, 215), (215, 216), (216, 210), (217, 218), (217, 219), (218, 220), (219, 226), (220,
221), (221, 227), (222, 223), (222, 224), (223, 211), (224, 228), (225, 229), (225, 230), (227, 238), (229, 231), (230,
241), (231, 232), (232, 233), (233, 234), (234, 242), (235, 236), (235, 237), (236, 226), (237, 243), (238, 228), (239,
240), (240, 244), (241, 245), (242, 247), (243, 248), (245, 246), (247, 243), (248, 249), (249, 250), (250, 251), (251,
252), (252, 253), (253, 254), (254, 244))
        check_maze_solver(G, 188)

        G = ((0, 1), (0, 2), (1, 3), (2, 38), (3, 4), (3, 5), (4, 6), (5, 39), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11),
(11, 12), (12, 13), (13, 40), (14, 15), (14, 16), (15, 17), (16, 41), (18, 19), (18, 20), (19, 21), (20, 42), (21, 22), (22,
23), (23, 24), (24, 25), (25, 26), (25, 27), (26, 28), (27, 43), (28, 29), (29, 44), (30, 31), (30, 32), (31, 33), (32, 45),
(33, 34), (33, 35), (34, 36), (35, 46), (36, 37), (37, 47), (39, 48), (40, 56), (42, 60), (44, 65), (45, 66), (46, 67), (48,
68), (49, 50), (50, 69), (51, 52), (51, 53), (52, 54), (53, 70), (54, 55), (55, 40), (56, 41), (57, 58), (57, 59), (58, 42),
```

```
97), (76, 77), (76, 78), (77, 68), (78, 100), (79, 70), (80, 101), (81, 82), (82, 102), (83, 84), (83, 85), (84, 86), (85, 103), (86, 87), (87, 71), (88, 89), (88, 90), (89, 91), (90, 104), (91, 92), (92, 72), (93, 94), (93, 95), (94, 73), (95, 105), (96, 106), (97, 98), (98, 99), (99, 107), (100, 108), (100, 109), (101, 112), (102, 116), (103, 117), (105, 128), (106, 131), (107, 133), (108, 110), (109, 134), (110, 111), (111, 135), (112, 136), (113, 114), (113, 115), (114, 102), (115, 137), (116, 138), (117, 118), (118, 119), (119, 139), (120, 121), (120, 122), (121, 104), (122, 140), (123, 124), (123, 125), (124, 126), (125, 141), (126, 127), (127, 142), (128, 143), (129, 130), (130, 106), (131, 132), (133, 144), (134, 145), (135, 146), (136, 147), (138, 148), (138, 149), (139, 151), (140, 152), (142, 155), (143, 156), (145, 163), (146, 164), (147, 137), (148, 150), (149, 165), (151, 166), (152, 153), (153, 154), (154, 167), (155, 143), (156, 157), (157, 158), (158, 159), (159, 160), (160, 161), (161, 162), (162, 144), (163, 168), (164, 169), (167, 179), (168, 196), (169, 170), (170, 171), (171, 197), (172, 173), (173, 165), (174, 175), (174, 176), (175, 166), (176, 198), (177, 178), (178, 199), (179, 200), (180, 181), (180, 182), (181, 183), (182, 201), (183, 184), (184, 185), (185, 186), (186, 187), (187, 188), (188, 202), (189, 190), (189, 191), (190, 192), (191, 203), (192, 193), (193, 194), (194, 195), (195, 204), (196, 205), (197, 208), (198, 213), (200, 216), (201, 217), (202, 222), (203, 223), (204, 227), (205, 206), (206, 207), (207, 228), (208, 209), (209, 210), (210, 211), (211, 212), (212, 198), (213, 214), (214, 215), (215, 199), (216, 229), (217, 218), (218, 219), (219, 220), (220, 221), (221, 230), (222, 231), (223, 232), (224, 225), (224, 226), (225, 204), (226, 233), (227, 234), (228, 237), (229, 251), (231, 257), (232, 258), (233, 259), (234, 260), (235, 236), (236, 261), (237, 262), (238, 239), (238, 240), (239, 241), (240, 263), (241, 242), (242, 243), (243, 244), (244, 245), (245, 246), (246, 267), (247, 248), (248, 249), (249, 250), (250, 264), (251, 265), (252, 253), (252, 254), (253, 255), (254, 266), (255, 256), (256, 230), (257, 267), (258, 268), (259, 269), (260, 270), (261, 271), (261, 272), (263, 273), (264, 283), (265, 284), (266, 285), (267, 291), (268, 292), (269, 293), (270, 294), (271, 262), (272, 295), (273, 296), (274, 275), (274, 276), (275, 277), (276, 297), (277, 278), (277, 279), (278, 280), (279, 298), (280, 281), (281, 282), (283, 299), (284, 266), (285, 300), (286, 287), (286, 288), (287, 289), (288, 301), (289, 290), (290, 302), (291, 303), (292, 304), (293, 305), (294, 306), (295, 307), (296, 310), (297, 311), (299, 317), (300, 320), (303, 321), (305, 322), (307, 308), (308, 309), (309, 323), (310, 324), (311, 325), (312, 313), (312, 314), (313, 315), (314, 326), (315, 316), (316, 299), (317, 318), (318, 319), (319, 327), (320, 301), (321, 304), (322, 328), (324, 332), (325, 333), (325, 334), (327, 341), (328, 352), (329, 330), (329, 331), (330, 323), (331, 355), (332, 356), (333, 335), (334, 357), (335, 336), (336, 326), (337, 338), (337, 339), (338, 340), (339, 358), (341, 342), (342, 343), (343, 344), (344, 345), (345, 346), (346, 347), (347, 348), (348, 349), (349, 350), (350, 351), (351, 359), (352, 353), (353, 354), (354, 360), (355, 361), (356, 364), (357, 365), (359, 383), (360, 386), (361, 387), (362, 363), (363, 388), (364, 389), (365, 366), (367, 368), (367, 369), (368, 358), (369, 390), (370, 371), (370, 372), (371, 373), (372, 391), (373, 374), (374, 375), (375, 376), (376, 377), (377, 378), (377, 379), (378, 380), (379, 392), (380, 381), (381, 382), (383, 384), (384, 385), (385, 360), (386, 393), (387, 394), (388, 395), (389, 396), (391, 404), (392, 409), (394, 419), (395, 420), (396, 397), (397, 398), (398, 421), (399, 400), (399, 401), (400, 390), (401, 422), (402, 403), (403, 391), (404, 405), (406, 407), (406, 408), (407, 392), (408, 423), (409, 424), (410, 411), (410, 412), (411, 413), (412, 425), (413, 414), (414, 415), (415, 416), (416, 417), (417, 418), (418, 393), (419, 426), (420, 427), (420, 428), (421, 430), (422, 431), (423, 442), (425, 443), (426, 453), (427, 429), (428, 454), (430, 455), (431, 456), (432, 433), (432, 434), (433, 435), (434, 457), (435, 436), (436, 437), (437, 438), (438, 458), (439, 440), (439, 441), (440, 423), (441, 459), (442, 460), (443, 461), (444, 445), (445, 462), (446, 447), (446, 448), (447, 449), (448, 463), (449, 450), (450, 451), (451, 452), (452, 464), (453, 465), (454, 466), (456, 470), (458, 473), (459, 474), (462, 478), (464, 482), (465, 483), (466, 484), (467, 468), (467, 469), (468, 455), (469, 485), (470, 457), (471, 472), (472, 486), (473, 487), (474, 488), (475, 476), (475, 477), (476, 461), (477, 489), (478, 463), (479, 480), (479, 481), (480, 464), (481, 490), (482, 491), (483, 492), (485, 493), (486, 500), (488, 501), (489, 504), (492, 484), (493, 494), (494, 495), (495, 496), (496, 497), (497, 498), (498, 499), (499, 486), (500, 487), (501, 502), (502, 503), (503, 489), (504, 505), (505, 506), (506, 507), (507, 508), (508, 509), (509, 510), (510, 490))

check_maze_solver(6, 255)

def test_sumas(self):
    def check_sumas(n, sz):
        S = sumas(n)
        self.assertEqual(len(S), sz, 'sumas({}) deberia tener {} secuencias'.format(n,sz))
        L = set(tuple(sorted(e)) for e in S)
        self.assertEqual(len(L), sz, 'sumas({}) deberia tener {} secuencias diferentes'.format(n,sz))
        for e in L:
            self.assertEqual(sum(e), n, 'La suma de {} no da {}'.format(e,n))

        check_sumas(4,5)
        check_sumas(5,7)
        check_sumas(6,11)
        check_sumas(7,15)

def test_planifica(self):
    m, k = 2, 25
    def check_planifica(L, n):
        D = planifica(L, m, k)
        self.assertEqual(sum(len(i) for i in D), n, 'planifica({}, {}, {}) debe guardar {} archivos. Devolvió {}'.format(L,m,k,n,D))
        files = set(sum(D, tuple()))
        self.assertEqual(len(files), n, 'planifica({}, {}, {}) debe guardar {} archivos diferentes'.format(L,m,k,n))
        check_planifica([10,15,20,8], 3)
        check_planifica([10,15], 2)
        check_planifica([10,25,15], 3)
        check_planifica([10,25,15,1], 3)
        check_planifica([10,1,2,3,15,4,25,15,1], 7)

def test_num_tanros(self):
    R = 25
    LL = (([10,15,20,8], 3), ([1,3,10,15,4,10], 2), ([10,24,16,19], 4))
    for L, nt in LL:
```

```
# Si usas Jupyter o VSCode descomenta la última línea
# Si usas IDLE, Python o PyCharm descomenta la penúltima
# main()
#main(argv=['first-arg-is-ignored'], exit=False)
```

Estado de la entrega

Número del intento	Este es el intento 1.
Estado de la entrega	No entregado
Estado de la calificación	Sin calificar
Fecha de entrega	sábado, 15 de junio de 2019, 00:00:00
Tiempo restante	141 días 10 horas
Última modificación	-
Agregar entrega	

Todavía no has realizado una entrega

[◀ Tema 7. Recursión y backtracking.](#)

Ir a...

[Tema 8. Programación dinámica ▶](#)

Usted se ha identificado como PABLO MANUEL MARTÍN ISABEL (Salir (29185) INFORMÁTICA

- Español – Internacional (es)
- English (en)
- Español – Internacional (es)

Descargar la app para dispositivos móviles